Functions in SQL Server

SQL Server functions are powerful tools that allow you to encapsulate logic and perform operations on data, returning a single result. They can be broadly categorized into several types, each serving a distinct purpose. This document provides an overview and examples of common SQL Server functions.

Scalar-Valued Functions

Scalar-valued functions process input parameters and return a single scalar value.

String Functions

These functions operate on string (text) data.

Function	Description	Example	Result
LEN()	Returns the length of the input string, excluding trailing spaces.	SELECT LEN('SQL Server');	10
LEFT()	Returns the left part of a string with the specified number of characters.	SELECT LEFT('Hello World', 5);	Hello
RIGHT()	Returns the right part of a string with the specified number of characters.	SELECT RIGHT('Hello World', 5);	World
SUBSTRING()	Extracts a substring from a string.	SELECT SUBSTRING('SQL Functions', 5, 9);	Functions
UPPER()	Converts a string to uppercase.	SELECT UPPER('sql server');	SQL SERVER
LOWER()	Converts a string to lowercase.	SELECT LOWER('SQL SERVER');	sql server

Function	Description	Example	Result
REPLACE()	Replaces all occurrences of a substring within a string.	SELECT REPLACE('Hello World', 'World', 'SQL');	Hello SQL
TRIM()	Removes leading and trailing spaces (or specified characters) from a string.	SELECT TRIM(' Trim Me ');	Trim Me
CONCAT()	Concatenates two or more string values.	SELECT CONCAT('SQL', ' ', 'Server');	SQL Server

Numeric Functions

These functions perform mathematical operations on numeric data.

Function	Description	Example	Result
ABS()	Returns the absolute (positive) value of a number.	SELECT ABS(-10.5);	10.5
CEILING()	Returns the smallest integer greater than or equal to the specified numeric expression.	SELECT CEILING(12.3);	13
FLOOR()	Returns the largest integer less than or equal to the specified numeric expression.	SELECT FLOOR(12.7);	12
ROUND()	Rounds a numeric value to a specified length or precision.	SELECT ROUND(123.456, 2);	123.46
POWER()	Returns the value of an expression to a specified power.	SELECT POWER(2, 3);	8
SQRT()	Returns the square root of a positive number.	SELECT SQRT(81);	9
RAND()	Returns a random float value between 0 and 1.	SELECT RAND();	(e.g., 0.123456789)

Date and Time Functions

These functions work with date and time data.

Function	Description	Example	Result (on at)
GETDATE()	Returns the current date and time of the system.	SELECT GETDATE();	Date Calendar event
SYSDATETIME()	Returns the current system date and time with higher precision than GETDATE().	SELECT SYSDATETIME();	Date Calendar event
DATEADD()	Adds a specified number of a datepart (e.g., day, month, year) to a date.	SELECT DATEADD(day, 7, GETDATE());	(current date + 7 days)
DATEDIFF()	Returns the count of specified datepart boundaries crossed between two specified dates.	SELECT DATEDIFF(day, '2023-01-01', '2023-01-10');	9
DATENAME()	Returns a string representing the specified datepart of the specified date.	SELECT DATENAME(month, GETDATE());	October
DATEPART()	Returns an integer representing the specified datepart of the specified date.	SELECT DATEPART(month, GETDATE());	10
YEAR()	Returns the year as an integer for a specified date.	SELECT YEAR(GETDATE());	2025
MONTH()	Returns the month as an integer for a specified date.	SELECT MONTH(GETDATE());	10
DAY()	Returns the day of the month as an integer for a specified date.	SELECT DAY(GETDATE());	8

Conversion Functions

These functions convert expressions from one data type to another.

Function	Description	Example	Result
CAST()	Converts an expression of one data type to another.	SELECT CAST('123' AS INT);	123
CONVERT()	Converts an expression of one data type to another, with an optional style parameter.	SELECT CONVERT(VARCHAR(10), GETDATE(), 101);	10/08/2025
PARSE()	Converts a string value to a specified numeric or date/time type. Requires a culture parameter.	SELECT PARSE('€123.45' AS MONEY USING 'en-US');	123.45
TRY_CAST()	Attempts to cast an expression to a specified data type. Returns NULL if the cast fails.	SELECT TRY_CAST('abc' AS INT);	NULL
TRY_CONVERT()	Attempts to convert an expression to a specified data type. Returns NULL if the conversion fails.	SELECT TRY_CONVERT(INT, 'xyz');	NULL

Logical Functions

These functions perform logical evaluations.

Function	Description	Example	Result
IIF()	Returns one of two values, depending on whether the Boolean expression evaluates to true or false.	SELECT IIF(10 > 5, 'Greater', 'Smaller');	Greater
CHOOSE()	Returns the item at the specified index from a list of values.	SELECT CHOOSE(2, 'Apple', 'Banana', 'Cherry');	Banana

Aggregate Functions

Aggregate functions perform calculations on a set of rows and return a single summary value. These are often used with the GROUP BY clause.

Function	Description	Example (using a table Products with Price column)	Result
COUNT()	Returns the number of items in a group.	SELECT COUNT(ProductID) FROM Products;	(e.g., 100)
SUM()	Calculates the sum of all or distinct values in an expression.	SELECT SUM(Price) FROM Products;	(e.g., 5000.00)
AVG()	Calculates the average of all or distinct values in an expression.	SELECT AVG(Price) FROM Products;	(e.g., 50.00)
MIN()	Returns the minimum value in a group.	SELECT MIN(Price) FROM Products;	(e.g., 10.00)
MAX()	Returns the maximum value in a group.	SELECT MAX(Price) FROM Products;	(e.g., 200.00)

User-Defined Functions (UDFs)

UDFs allow you to create custom functions to encapsulate complex logic that can be reused throughout your database.

Scalar UDF

Returns a single scalar value.CREATE FUNCTION GetFullName (@FirstName VARCHAR(50), @LastName VARCHAR(50))

RETURNS VARCHAR(101)

AS

BEGIN

```
    RETURN @FirstName + ' ' + @LastName;

END;
```

Example Usage:SELECT dbo.GetFullName('John', 'Doe');

Result: John Doe

Table-Valued UDF (TVF)

Returns a table data type.

Inline Table-Valued Function (ITVF)

Returns a single SELECT statement.CREATE FUNCTION GetProductsByCategory (@CategoryID INT)

```
RETURNS TABLE

AS

RETURN

(

    SELECT ProductID, ProductName, Price

    FROM Products

    WHERE CategoryID = @CategoryID

);
```

Example Usage:SELECT * FROM dbo.GetProductsByCategory(1);

Multi-Statement Table-Valued Function (MSTVF)

Returns a table variable and can contain multiple statements.CREATE FUNCTION GetCustomersByCity (@City VARCHAR(100))

```
RETURNS @Customers TABLE
```

```
(
    CustomerID INT,
    CustomerName VARCHAR(100),
    City VARCHAR(100)
)
AS
BEGIN
    INSERT INTO @Customers (CustomerID, CustomerName, City)
    SELECT CustomerID, CustomerName, City
    FROM Customers
    WHERE City = @City;
    RETURN;
END;
```

Example Usage:SELECT * FROM dbo.GetCustomersByCity('London');

This document provides a foundational understanding of various function types in SQL Server with practical examples. For more in-depth information, refer to the official SQL Server documentation.